

PROJET M5 — FORMATION CONSULTANT SAP ERP (SD)

Rapport de résolution

Ticket utilisateur — Rapport de stock vide

ZMM_GET_MAT_STOCK_PLANT → ZSPRO_RLEGROS_ZMM_GET_MAT

CONSULTANT SAP RLEGROS

DEMANDEUR Sylvain Goebels — Service Planification

PACKAGE ZP_SPRO_RLEGROS

DATE 31 juillet 2026

Contexte

J'interviens sur le programme ZMM_GET_MAT_STOCK_PLANT, que j'ai développé à partir d'une spécification de reporting que j'avais rédigée en amont, pour consulter le stock disponible par matériel et par magasin (plant). Ce programme est utilisé par le service Planification pour organiser la livraison client.

Problème signalé

J'ai reçu de Sylvain Goebels un signalement indiquant que le programme ne retournait **aucun résultat**, alors que du stock existait bien pour le matériel testé.

Paramètres utilisés lors du signalement :

- Material : EWMS4-01
- Plant : 1210
- Warehouse number : (vide)

Résultat obtenu : tableau de résultats entièrement vide, alors que le service Planification confirmait disposer de stock sur ce matériel.

Méthodologie appliquée

Conformément à la consigne du projet, j'ai suivi la méthodologie en 5 étapes suivante pour résoudre ce ticket :

1. Reproduire le problème
2. Identifier la cause via debug
3. Créer un programme de correction (ZSPRO_RLEGROS_ZMM_GET_MAT) dans le package personnel
4. Copier le programme original dans ce nouveau programme
5. Corriger le programme copié et valider le comportement

Étape 1 — Reproduction du problème

J'ai exécuté le programme original ZMM_GET_MAT_STOCK_PLANT avec les paramètres exacts communiqués par Sylvain Goebels, afin de confirmer le comportement défaillant avant toute investigation.

Paramètres saisis :

Material:	EWMS4-01	to:	
Plant:	1210	to:	
Storage Location:		to:	

Résultat obtenu : j'ai bien reproduit le problème — aucune donnée n'est retournée.

Program ZMM_GET_MAT_STOCK Aucune donnée trouvée

Résultat vide reproduisant le bug

Je confirme que le problème est reproductible et qu'il n'est pas lié à une erreur de saisie de la part de l'utilisateur.

Contrôle complémentaire — Le stock existe-t-il réellement ?

Pourquoi ce contrôle : avant d'aller plus loin dans mon investigation, j'ai voulu écarter une explication simple — et si le matériel n'avait tout simplement aucun stock ? Un rapport vide serait alors un comportement normal, pas un bug.

Méthode : j'ai réalisé une vérification indépendante du stock réel via la transaction MMBE (Stock Overview), en dehors du programme mis en cause.

Selection

Material: EWS4-01

Small Part, Slow-Moving Item

Material Type: HAWA

Trading Goods

Unit of Measure: PC

Base Unit of Measure

Stock Overview

▼

▲

🔍

📄

▼

🔍 Detailed Display

Client/Company Code/Plant/Storage Location/Batch/Special Stock	Unrestricted use	Qual. inspection
▼ 📁 Full	8,634.000	
▼ 📁 1210 G+ FR	8,634.000	
▼ 📁 1210 Paris	8,634.000	
📁 121A Finished Product	1,002.000	
📁 121D Received on Dock	2.000	
📁 121S EWM A. for Sale	18.000	
📁 202 ATATCHEUR STO LO	130.000	
📁 5005 TEKOL KEVO	220.000	
📁 9009 ?	30.000	
📁 EG01 EG_TEKOL	145.000	
📁 FH01 FHAUSER STO LOC	15.000	
📁 HP01 HPYCKAERT STO LO	100.000	
📁 NC01 NCAUTELA STO LOC	165.000	
📁 RT01 RT tekol storage	147.000	
📁 T100 Tekol WM	227.000	
📁 TE01 WHONORE Storage	130.000	
📁 TOS Tekol osanli	130.000	
▼ 📁 W001 Storage WM	6,058.000	
Customer Stock	300.000	
📁 W003 Storage KBA	15.000	
📁 ZTSL TEKOL Sto.Loc	100.000	

Transaction MMBE — Stock Overview pour EWMS4-01, plant 1210

Résultat : j'ai constaté 8 634 unités disponibles au total pour le matériel EWMS4-01 sur le plant 1210, réparties sur 17 emplacements de stockage. Le stock existe donc bel et bien — je confirme que le rapport vide constaté dans le ticket est un dysfonctionnement du programme, et non une absence légitime de données.

Étape 2 — Identification de la cause (debug)

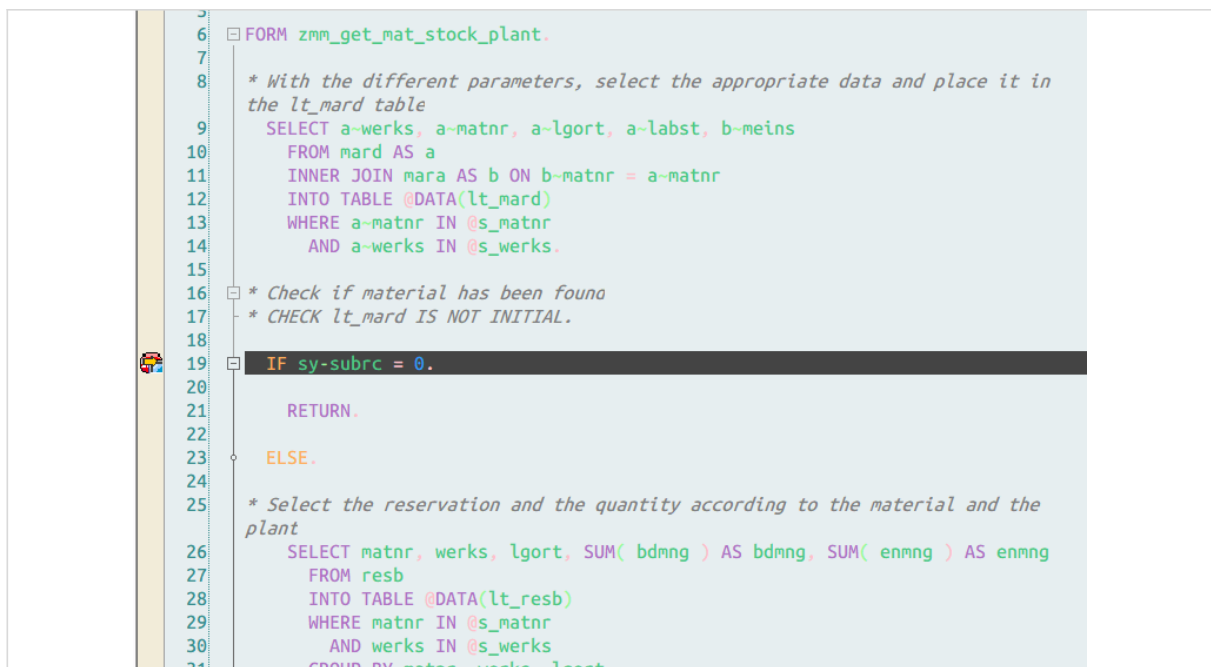
En prenant connaissance du code pour la première fois, un élément m'a immédiatement interpellé : la suite de l'exécution du programme repose entièrement sur la valeur d'une seule variable, SY-SUBRC, testée juste après le SELECT. Cette dépendance m'a conduit à m'intéresser de plus près à ce que représente concrètement cette variable, ainsi qu'au fonctionnement du mot-clé RETURN qui l'exploite dans cette structure conditionnelle.

SY-SUBRC : cette variable système, renseignée automatiquement par le runtime ABAP après de nombreuses instructions (SELECT, PERFORM, CALL METHOD, opérations arithmétiques...), indique si l'instruction précédente s'est déroulée avec succès. Une valeur à 0 signale un succès, une valeur différente de 0 signale un échec ou l'absence de résultat. Pour un SELECT ... INTO TABLE en particulier, cela se traduit par : 0 dès qu'au moins une ligne a été trouvée, et une autre valeur (généralement 4) lorsqu'aucune ligne ne correspond aux critères.

RETURN : ce mot-clé interrompt immédiatement l'exécution de la routine (FORM) en cours et rend la main au programme appelant, sans exécuter les instructions suivantes de cette routine. Combiné à un test sur SY-SUBRC, il permet typiquement de sortir prématurément d'un traitement lorsque la donnée attendue n'a pas été trouvée.

C'est précisément cette combinaison — la poursuite du programme conditionnée au sens d'un test sur SY-SUBRC, associée à une sortie anticipée via RETURN — qui a retenu mon attention dès la première lecture : ce type de structure mérite toujours une vérification attentive du sens exact du test.

J'ai donc mené un debug pas-à-pas sur la FORM get_mat_stock_plant, en posant un breakpoint sur le test conditionnel suivant la sélection des données de stock (SELECT ... FROM mard ... INTO TABLE @DATA(lt_mard)).



```
5
6  FORM zmm_get_mat_stock_plant.
7
8      * With the different parameters, select the appropriate data and place it in
      * the lt_mard table
9      SELECT a-werks, a-matnr, a-lgort, a-labst, b-meins
10     FROM mard AS a
11     INNER JOIN mard AS b ON b-matnr = a-matnr
12     INTO TABLE @DATA(lt_mard)
13     WHERE a-matnr IN @s_matnr
14           AND a-werks IN @s_werks.
15
16  * Check if material has been found
17  * CHECK lt_mard IS NOT INITIAL.
18
19  IF sy-subrc = 0.
20
21      RETURN.
22
23  ELSE.
24
25      * Select the reservation and the quantity according to the material and the
      * plant
26      SELECT matnr, werks, lgort, SUM( bdmng ) AS bdmng, SUM( enmng ) AS enmng
27     FROM resb
28     INTO TABLE @DATA(lt_resb)
29     WHERE matnr IN @s_matnr
30           AND werks IN @s_werks
31     GROUP BY matnr, werks, lgort.
```

Breakpoint sur le test sy-subrc

St...	Variable	Val...	Val.	Ch...	Hexadecimal Value
☐	SY-SUBRC		0	☐	00000000
☐	LT_MARD		[18x5(110)]Standard Table		

Valeurs des variables SY-SUBRC et LT_MARD

Constat :

- SY-SUBRC = 0 (le SELECT a bien trouvé des résultats)
- LT_MARD contient effectivement des lignes de stock (18 entrées)

Or le code testait :

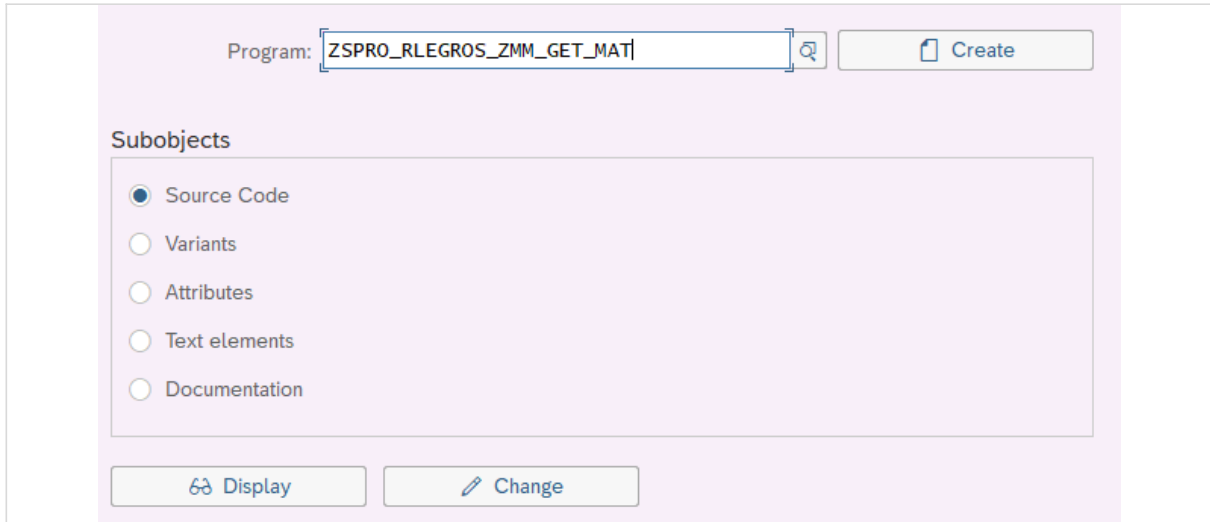
```
IF sy-subrc = 0.
    RETURN.
ELSE.
    " ... logique de calcul et remplissage du résultat final ...
ENDIF.
```

Cause racine identifiée : j'ai déterminé que la condition est inversée. En ABAP, SY-SUBRC = 0 signifie que l'instruction précédente (ici le SELECT) a réussi, donc que des données ont été trouvées — ce n'est donc pas un cas d'erreur. Avec cette condition inversée, le programme sort de la routine (RETURN) précisément quand des données existent, et n'entre dans le bloc de calcul (ELSE) que lorsqu'il n'y a rien à traiter. Dans les deux cas de figure, le tableau de résultat final reste vide.

J'ai remarqué qu'une ligne commentée dans le code (* CHECK lt_mard IS NOT INITIAL.) suggère qu'il s'agit d'une régression par rapport à une version antérieure du programme, où le contrôle était correctement formulé.

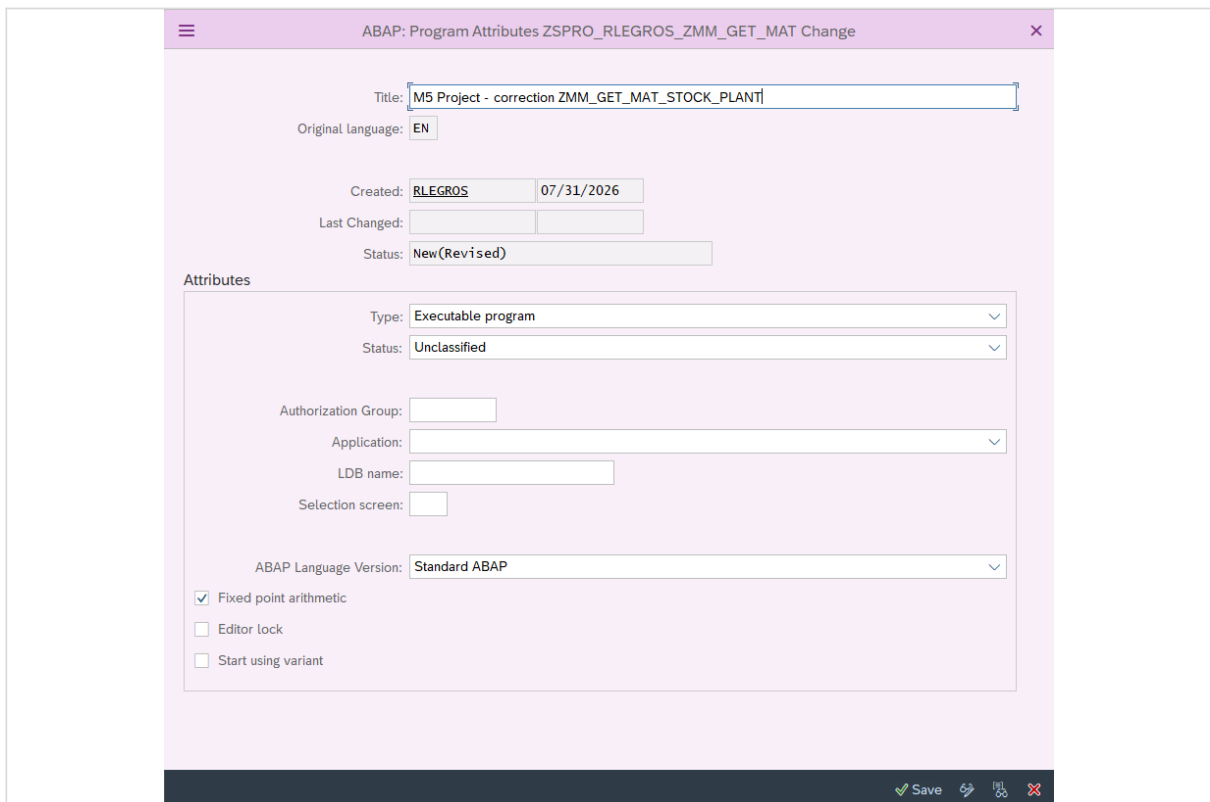
Étape 3 — Création du programme de correction

J'ai créé un nouveau programme exécutable dans mon package personnel ZP_SPRO_RLEGROS, conformément à la convention de nommage imposée (ZSPRO_**_ZMM_GET_MAT) :



The screenshot shows the 'Create' dialog in SAP. The 'Program' field is filled with 'ZSPRO_RLEGROS_ZMM_GET_MAT'. Below it, the 'Subobjects' section has 'Source Code' selected. At the bottom, there are 'Display' and 'Change' buttons.

Création du programme ZSPRO_RLEGROS_ZMM_GET_MAT



The screenshot shows the 'Program Attributes' dialog for the program 'ZSPRO_RLEGROS_ZMM_GET_MAT'. The 'Title' is 'M5 Project - correction ZMM_GET_MAT_STOCK_PLANT'. The 'Original language' is 'EN'. The 'Created' date is '07/31/2026'. The 'Status' is 'New(Revised)'. Under 'Attributes', the 'Type' is 'Executable program' and the 'Status' is 'Unclassified'. The 'ABAP Language Version' is 'Standard ABAP'. There are checkboxes for 'Fixed point arithmetic' (checked), 'Editor lock', and 'Start using variant'. At the bottom right, there are 'Save', 'Cancel', and 'Help' buttons.

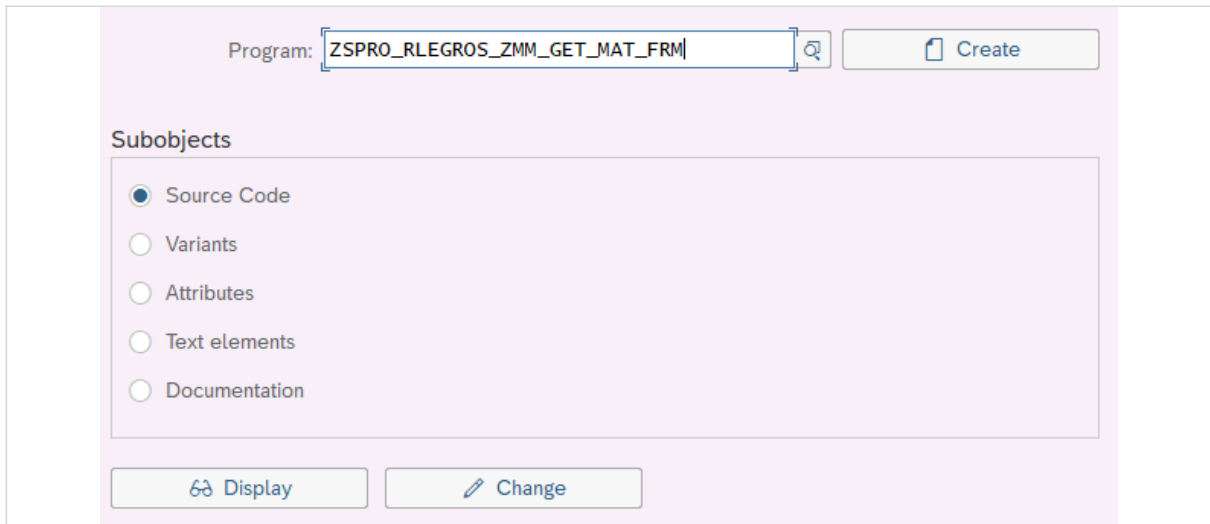
Attributs du programme créé

Repository Browser		
Package		
ZP_SPRO_RLEGROS		
← → ⌵ ⌶ 🔍 📁 ★ ↻		
	Object Name	Description
☐	✓  ZP_SPRO_RLEGROS	M5 Initiation ABAP
☐	✓  Programs	
☐	>  ZPP_SPRO_RLEGROS_1	First RLS program
☐	>  ZPP_SPRO_RLEGROS_2	Program ZPP_SPRO_RLEGROS_2
☐	>  ZSPRO_RLEGROS_ZMM_GET_MAT	M5 Project - correction ZMM_GET_MAT_STOCK_PLANT
☐	>  Message Classes	

Programme visible dans l'arborescence du package

Étape 4 — Copie du programme original

Le programme original étant structuré en 3 INCLUDE distincts (déclarations de données, classe locale ALV, logique/FORM), j'ai reproduit fidèlement la même structure afin de rester cohérent avec l'organisation d'origine, en recréant chaque INCLUDE sous le nom correspondant (ZSPRO_RLEGROS_ZMM_GET_MAT_DAT, _LCL, _FRM).

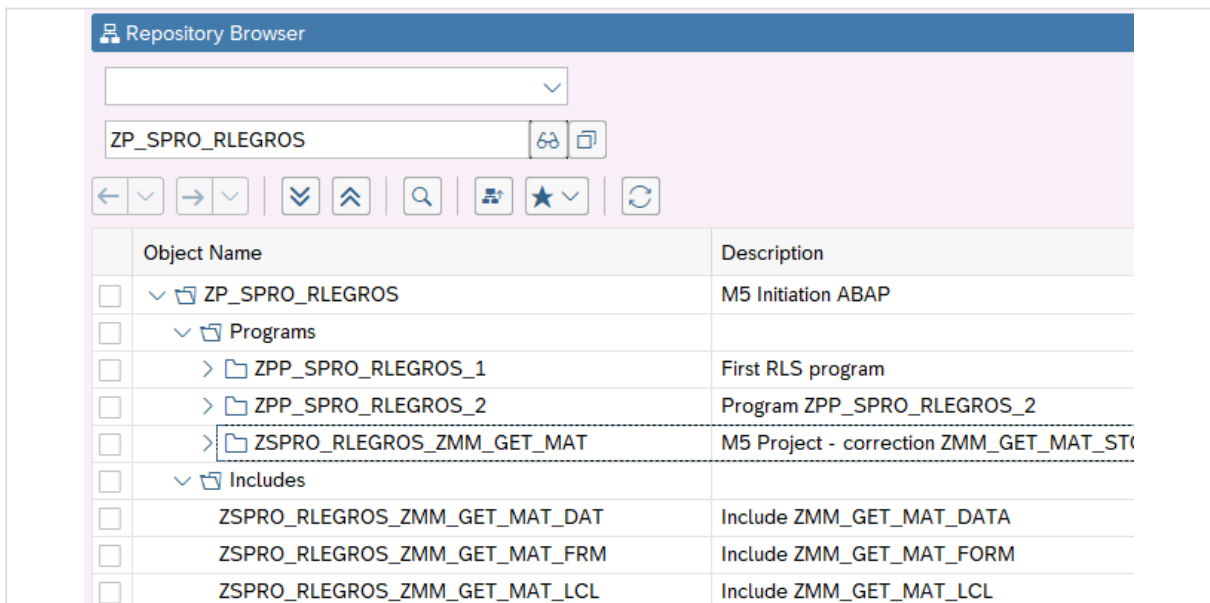


Program:

Subobjects

- ☒ Source Code
- ☐ Variants
- ☐ Attributes
- ☐ Text elements
- ☐ Documentation

Création d'un INCLUDE



Object Name	Description
✓ ZP_SPRO_RLEGROS	M5 Initiation ABAP
✓ Programs	
> ZPP_SPRO_RLEGROS_1	First RLS program
> ZPP_SPRO_RLEGROS_2	Program ZPP_SPRO_RLEGROS_2
> ZSPRO_RLEGROS_ZMM_GET_MAT	M5 Project - correction ZMM_GET_MAT_STC
✓ Includes	
ZSPRO_RLEGROS_ZMM_GET_MAT_DAT	Include ZMM_GET_MAT_DATA
ZSPRO_RLEGROS_ZMM_GET_MAT_FRM	Include ZMM_GET_MAT_FORM
ZSPRO_RLEGROS_ZMM_GET_MAT_LCL	Include ZMM_GET_MAT_LCL

Les 3 INCLUDE recréés dans le package

J'ai copié à l'identique le contenu de chaque INCLUDE d'origine dans son équivalent, et j'ai mis à jour les instructions INCLUDE du programme principal ainsi que l'instruction REPORT pour pointer vers les nouveaux noms d'objets.

À ce stade, mon programme copié reproduit le même comportement défaillant que l'original — ce qui est attendu, puisqu'aucune correction n'a encore été appliquée.

Étape 5 — Correction et validation

J'ai appliqué la correction exclusivement dans l'INCLUDE
ZSPRO_RLEGROS_ZMM_GET_MAT_FRM, en inversant la condition fautive :

```
IF sy-subrc <> 0.  
  RETURN.  
ELSE.  
  " ... logique de calcul et remplissage du résultat final ...  
ENDIF.
```

```
5  
6  FORM zmm_get_mat_stock_plant.  
7  
8  * With the different parameters, select the appropriate data and place it in the lt_mard table  
9  SELECT a~werks, a~matnr, a~lgort, a~labst, b~meins  
10 FROM mard AS a  
11 INNER JOIN mara AS b ON b~matnr = a~matnr  
12 INTO TABLE @DATA(lt_mard)  
13 WHERE a~matnr IN @s_matnr  
14 AND a~werks IN @s_werks.  
15  
16 * Check if material has been found  
17 * CHECK lt_mard IS NOT INITIAL.  
18  
19 IF sy-subrc <> 0.  
20  
21 RETURN.  
22  
23 ELSE.  
24
```

Code corrigé

J'ai réalisé un test de validation avec les mêmes paramètres que le ticket initial :

Material:	EWMS4-01	to:	
Plant:	1210	to:	
Storage Location:		to:	

Paramètres de test

Résultat obtenu après correction :

Plant	Material	Location	Unrestricted	Quantity withdrawn	Unit
1210	EWMS4-01	121D	2	0	PC
1210	EWMS4-01	121S	18	0	PC
1210	EWMS4-01	W001	6,058	2	PC
1210	EWMS4-01	FH01	15	2	PC
1210	EWMS4-01	HP01	100	2	PC
1210	EWMS4-01	NC01	165	2	PC
1210	EWMS4-01	TE01	130	2	PC
1210	EWMS4-01	121A	1,002	2	PC
1210	EWMS4-01	W003	15	2	PC
1210	EWMS4-01	TOS	130	2	PC
1210	EWMS4-01	RT01	147	2	PC
1210	EWMS4-01	202	130	2	PC
1210	EWMS4-01	ST01	0	2	PC
1210	EWMS4-01	EG01	145	2	PC
1210	EWMS4-01	T100	227	2	PC
1210	EWMS4-01	ZTSL	100	2	PC
1210	EWMS4-01	9009	30	2	PC
1210	EWMS4-01	5005	220	2	PC

Résultat après correction — stock affiché

Je constate que le programme retourne désormais correctement les 18 lignes de stock disponibles pour le matériel EWMS4-01 sur le plant 1210, réparties par magasin, avec les quantités disponibles (unrestricted) et réservées (quantity withdrawn) correctement calculées.

Synthèse de l'analyse de cause racine

Symptôme	Le rapport de stock retourne systématiquement un tableau vide, quel que soit le matériel/plant interrogé.
Cause	Inversion de la condition de contrôle après le SELECT : IF sy-subrc = 0 au lieu de IF sy-subrc <> 0.
Effet	Quand des données sont trouvées (sy-subrc = 0), le programme sort de la routine avant tout calcul (RETURN) ; quand aucune donnée n'est trouvée, le bloc de calcul est exécuté mais sur une table vide. Dans les deux cas, aucun résultat n'est jamais produit.
Correction	Inversion du test : IF sy-subrc <> 0.
Portée	Modification d'une seule ligne, sans impact sur le reste de la logique (sélection des réservations, calcul du disponible, affichage ALV).

Conclusion et recommandation

J'ai résolu et validé le problème signalé par Sylvain Goebels sur la copie de développement ZSPRO_RLEGROS_ZMM_GET_MAT.

Recommandation : j'estime nécessaire d'appliquer la même correction (inversion de la condition sy-subrc) directement sur le programme de production ZMM_GET_MAT_STOCK_PLANT, après revue et transport selon le processus de gestion des changements en vigueur, afin que le service Planification retrouve un rapport de stock fonctionnel.